An Applied Agentic AI System for Autonomous Literature Review and Research Planning

Project Interim Report

Rajnish Sahani, Deven Wagh, Hangye Li, Yonghao Li
rsahani@syr.edu | dpwagh@syr.edu | hli299@syr.edu | yli598@syr.edu
March 20, 2026

***Abstract* —** This report presents the midterm status of an applied agentic AI system designed to autonomously conduct academic literature reviews and generate structured research plans. The system employs a modular multi-agent architecture built on LangGraph, comprising four specialized agents Search, Screening, Synthesis, and Planning — coordinated through a shared state and an adaptive feedback loop. At the midterm milestone, all core agents and the orchestration layer are fully implemented. This report documents the system architecture, implementation decisions, technical challenges encountered, and the evaluation plan for the final submission.

1. INTRODUCTION

The exponential growth of scientific literature in computer science and artificial intelligence has created a significant challenge for researchers. Thousands of papers are published monthly across journals, conferences, and preprint platforms, making it increasingly difficult to maintain awareness of prior work. The literature review process one of the most foundational stages of any research project has become time-consuming, inconsistent, and cognitively demanding, particularly for early-stage graduate students who lack systematic strategies for navigating large bodies of literature.

Recent advances in large language models and autonomous agent frameworks present new opportunities to augment this workflow. Agentic AI systems differ fundamentally from static pipelines by emphasizing goal-directed behavior, autonomous planning, tool use, reflection, and adaptation. Rather than

executing a fixed sequence of steps, agentic systems dynamically determine what actions to take based on intermediate results.

This project frames literature review and research planning as an applied domain for studying agentic AI systems. The system accepts a research topic as input and autonomously produces a curated list of relevant papers, structured summaries, identified themes, research gaps, and a concrete research plan. The focus is not on developing new language models, but on system-level architecture coordinating specialized agents over shared state to achieve a complex cognitive goal.

2. PROJECT OBJECTIVES

The primary objectives of this project, as stated in the original proposal, are as follows:

- Design a modular multi-agent architecture that decomposes the literature review workflow into specialized autonomous roles
- Enable agents to make informed decisions regarding query generation, source selection, relevance screening, synthesis, and planning
- Integrate existing Python libraries, academic APIs, and large language models as external tools
- Implement an adaptive decision loop allowing agents to refine strategies based on intermediate results
- Evaluate the effectiveness, adaptability, and transparency of the system compared to static baseline workflows

At the midterm stage, objectives 1 through 3 are substantially complete, and the adaptive loop (objective 4) is implemented and functional. Quantitative evaluation (objective 5) remains the primary work for the final phase.

3. SYSTEM ARCHITECTURE

3.1 Overview

The system is built on LangGraph, a framework for constructing stateful, cyclic multi-agent workflows using a directed graph model. The pipeline consists of four specialized agents coordinated by a central orchestrator. All agents communicate through a shared TypedDict state object that is passed through the graph and updated at each node. This design ensures full transparency every intermediate result is recorded in the shared state and accessible for inspection and evaluation.

3.2 Shared State Schema

The AgentState TypedDict, defined in state/shared_state.py, serves as the single source of truth for the entire pipeline. Table 1 describes its fields.

Field	Type	Description
research_topic	str	Input research topic provided by the user
search_queries	List[str]	LLM-generated search queries
raw_papers	List[Paper]	All papers retrieved from arXiv
screened_papers	List[Paper]	Papers passing relevance threshold (score ≥ 0.5)
summaries	List[str]	LLM-generated 2–3 sentence summaries per paper
themes	List[str]	3–5 cross-paper themes identified by LLM
research_gaps	List[str]	Gaps in existing literature identified by LLM
research_plan	str	Structured plan with questions, methodology, evaluation
iteration / max_iterations	int	Feedback loop control counters
messages	List[str]	Agent activity log for transparency

Table 1: AgentState schema fields and descriptions.

3.3 Agent Descriptions

Search Agent

The Search Agent is the entry point of the pipeline. Given the research topic, it uses the LLM with a carefully designed prompt to generate three diverse and specific search queries. For each query, it calls both the arXiv API and the Semantic Scholar API to retrieve up to five papers per source, which are then deduplicated by title across both sources. Generating multiple queries rather than a single one significantly improves coverage, since different phrasings surface different subsets of the literature a design directly informed by the ReAct pattern of reasoning before acting.

When the LLM returns a malformed or unparseable query list — for example, returning plain text instead of a Python list the agent falls back gracefully to using the raw research topic as a single query. Similarly, if either retrieval API is unavailable or returns an error, the agent catches the exception, logs it to the messages field in shared state, and continues with results from the other source. This ensures the pipeline never halts due to a single tool failure.

Screening Agent

The Screening Agent evaluates each paper retrieved by the Search Agent. For every paper, it sends the title and a 500-character excerpt of the abstract to the LLM with a prompt requesting a relevance score between 0.0 and 1.0. Papers scoring below 0.5 are filtered out; remaining papers are sorted by descending relevance. Using LLM-based semantic scoring rather than keyword matching allows the system to assess conceptual relevance, which is especially important for interdisciplinary topics where exact terminology may differ across subfields.

When the LLM returns a score that cannot be parsed as a float — due to added explanation text or unexpected formatting the agent assigns a default score of 0.5, ensuring the paper is retained for human review rather than silently discarded. This conservative fallback prevents information loss under ambiguous model outputs. Conflicting scores across papers are handled implicitly through ranking: papers are sorted by score, so downstream agents always operate on the most confidently relevant material first.

Synthesis Agent

The Synthesis Agent generates a 2–3 sentence summary for each screened paper using the LLM, capturing key contributions and methodology. It then sends all summaries together to the LLM to identify 3–5 major cross-cutting themes. Separating summarization from theme identification allows each task to be focused and reliable; theme identification over the full set of summaries provides sufficient comparative context for the model.

If the LLM returns theme output that cannot be parsed as a Python list for instance, a numbered list in plain text the agent catches the parse error and substitutes a single generic theme ("General AI Research") rather than raising an exception. This ensures the pipeline always produces a complete output even when individual LLM responses deviate from the expected format.

Planning Agent

The Planning Agent takes the identified themes and produces two outputs through sequential LLM calls: first, a list of 3–5 research gaps not addressed by existing literature; second, a structured research plan comprising proposed research questions, suggested methodology, and an evaluation strategy. Separating gap identification from plan generation ensures the resulting plan is explicitly grounded in identified gaps rather than produced generically from the topic alone.

If the gaps response cannot be parsed, the agent substitutes a placeholder gap and proceeds to plan generation rather than halting. This design decision reflects a deliberate tradeoff: a partial but complete pipeline output is more useful for evaluation and demonstration purposes than a failed run with no output at all.

3.4 Orchestrator and Feedback Loop

The orchestrator (controller/orchestrator.py) constructs a LangGraph StateGraph connecting all agents. Following the screening step, a conditional edge evaluates the quality of intermediate output: if fewer than three papers passed screening and iterations remain below the configured maximum, the graph routes back to the Search Agent for a revised attempt with different queries. Otherwise, it proceeds to synthesis and planning. This loop implements the core agentic principle of adaptive behavior the system evaluates its own intermediate output and decides whether additional search effort is warranted before committing to synthesis.

Node	Agent Called	Role

search	search_agent()	Query generation and arXiv retrieval
screen	screening_agent()	Relevance scoring and filtering
increment	increment_iteration()	Iteration counter update
synthesize	synthesis_agent()	Summarization and theme identification
plan	planning_agent()	Gap analysis and research plan generation

Table 2: LangGraph nodes and their roles in the pipeline.

4. IMPLEMENTATION DETAILS

4.1 Technology Stack

The system is implemented in Python 3.11. LangGraph serves as the orchestration framework, chosen for its native support for stateful cyclic graphs, shared state management, and conditional routing. The arXiv Python API provides paper retrieval without requiring authentication, offering broad coverage of computer science and AI literature. The full dependency set is managed within a Python virtual environment for reproducibility.

Component	Technology	Rationale
Agent Orchestration	LangGraph	Stateful cyclic graphs, conditional routing, native LangChain integration
LLM Provider	Google Gemini (under evaluation — see §4.2)	Free tier available; provider-agnostic architecture allows substitution
Paper Retrieval	arXiv Python API	Free, no authentication required, broad CS/AI coverage
State Management	Python TypedDict	Type-safe schema compatible with LangGraph's state system
Runtime	Python 3.11 + virtualenv	LangGraph requires Python 3.10+; isolated dependencies

Table 3: Technology stack with rationale.

4.2 LLM Provider: Current Status and Reasoning

The selection of a production LLM provider is an active decision the team is finalizing. Our original proposal referenced OpenAI GPT-4o-mini as the candidate model. During implementation, we evaluated multiple providers against three criteria: cost, API accessibility, and rate limits suitable for a research prototype running multiple sequential agent calls per execution.

A key architectural decision was made early: the LLM is instantiated inside each agent function rather than at module import time. This design ensures the system is fully provider-agnostic switching providers requires changing a single line per agent file. It also resolved a technical issue where module-level LLM initialization was occurring before environment variables were loaded.

Provider	Model	Free Tier	Notes
OpenAI	gpt-4o-mini	No	Most stable; well-documented behavior; requires billing
Google Gemini	gemini-2.0-flash	Limited daily quota	Currently integrated; quota constraints encountered during development testing
Groq	llama-3.1-70b	Generous limits	Strong candidate for development phase; fast inference
Anthropic	claude-haiku-3.5	No	High output quality; suitable for production evaluation phase

Table 4: LLM provider evaluation.

During development, the Google Gemini free tier daily quota was exhausted due to the high volume of sequential API calls generated across four agents during debugging and iteration. This is expected behavior for any free-tier provider under multi-agent workloads. The system has been verified to initialize correctly and route the API key properly; the issue is operational rather than architectural. For the final evaluation phase, we will use either OpenAI gpt-4o-mini or Groq, both of which provide sufficient throughput for the evaluation runs required.

4.3 Implementation Status

Module	Status	Description
state/shared_state.py	Complete	TypedDict schema for Paper and AgentState
tools/arxiv_tool.py	Complete	arXiv API wrapper returning Paper objects
agents/search_agent.py	Complete	LLM query generation + arXiv retrieval
agents/screening_agent.py	Complete	LLM relevance scoring + filtering
agents/synthesis_agent.py	Complete	LLM summarization + theme identification
agents/planning_agent.py	Complete	LLM gap identification + plan generation
controller/orchestrator.py	Complete	LangGraph graph with adaptive feedback loop
main.py	Complete	Entry point with formatted result output
tools/semantic_scholar.py	Pending	Secondary retrieval source for broader coverage

evaluation/metrics.py	Pending	Precision, coverage, efficiency metrics
notebooks/demo.ipynb	Pending	End-to-end demonstration notebook

Table 5: Implementation status by module.

5. CHALLENGES ENCOUNTERED

Several technical challenges were encountered during the implementation phase. Each was diagnosed and resolved, and the resolutions informed broader architectural decisions.

Challenge	Resolution
LangChain v0.3+ restructured prompt imports, breaking module loading	Updated all imports from langchain.prompts to langchain_core.prompts across all agent files
Module-level LLM initialization executed before dotenv loaded environment variables	Moved LLM instantiation inside each agent function; resolved key-not-found errors at import time
load_dotenv() failed to locate .env file when script was run from non-root directories	Adopted load_dotenv(find_dotenv()) which traverses parent directories automatically
Gemini free tier quota exhausted during development iteration	Architecture is provider-agnostic by design; final evaluation will use a provider with adequate throughput
LangGraph conditional routing initially incremented iteration incorrectly	Introduced a dedicated increment node to separate counter update logic from routing decision logic

Table 6: Challenges and resolutions.

6. EVALUATION PLAN

6.1 Evaluation Design

The evaluation compares the agentic system against a clearly defined static baseline: a single arXiv keyword search using only the raw research topic as the query, with no LLM-based screening, summarization, or planning. This baseline represents the most common manual starting point for a literature review and isolates the contribution of the agentic components.

Evaluation will be conducted on five pre-selected research topics spanning distinct subfields of AI, chosen to test generalization across domain vocabulary and paper density. The selected topics are: (1) neuro-symbolic AI, (2) retrieval-augmented generation, (3) multi-agent systems for task planning, (4) LLM alignment and safety, and (5) agentic AI for scientific discovery. For each topic, a ground-truth set of 10 known relevant papers was manually curated from Google Scholar by the team prior to running the system, ensuring the evaluation is not contaminated by system outputs.

Each topic will be run three times through both the agentic system and the baseline to account for non-determinism in LLM outputs. Results will be averaged across runs. Human relevance judgments will be made independently by two team members per topic; disagreements will be resolved by majority vote among all four team members.

6.2 Quantitative Metrics

The following metrics will be computed for both the agentic system and the baseline across all five topics:

Metric	Definition	Measurement Method	Target
Precision	Proportion of retrieved papers judged relevant	Human review by 2 team members per topic	> 0.70
Coverage	Proportion of ground-truth papers retrieved	Checked against 10 pre-curated papers per topic	> 0.60
API Efficiency	LLM calls per complete run	Counted from agent messages log	< 40 calls
Iteration Rate	Proportion of runs needing > 1 search iteration	Tracked via iteration counter in shared state	< 30%
Theme Quality	Relevance of identified themes to topic	Rated 1-5 by 2 independent reviewers	> 3.5 mean
Plan Actionability	Usefulness of generated research plan	Rated 1-5 by 2 independent reviewers	> 3.5 mean

Table 7: Quantitative evaluation metrics, measurement methods, and targets.

6.3 Qualitative Evaluation Protocol

Each team member will independently conduct a literature review on one assigned topic using the system output and rate the following dimensions on a structured 1-5 rubric: (1) coherence does the research plan follow logically from the identified gaps? (2) actionability — could a researcher realistically follow the proposed methodology? (3) transparency do the agent messages sufficiently explain why each decision was made? (4) novelty do the identified research gaps represent genuine open questions not already addressed in the retrieved papers?

Ratings will be collected in a shared spreadsheet and averaged. Inter-rater reliability will be reported using Cohen's kappa for the binary relevance judgments. All evaluation data, rubrics, and raw results will be included in the final report appendix to ensure reproducibility. The complete evaluation codebase will be committed to the GitHub repository under evaluation/metrics.py.

7. CONCLUSION

At the midterm milestone, the core agentic system architecture is fully designed and implemented. All four specialized agents, the LangGraph orchestrator, and the adaptive feedback loop are complete and functioning. The codebase is version-controlled on GitHub and structured for collaborative development across the team.

The primary remaining work consists of three items: finalizing the LLM provider for stable end-to-end evaluation runs, implementing the evaluation framework in evaluation/metrics.py, and integrating the Semantic Scholar API as a secondary retrieval source to improve paper coverage. The system architecture directly demonstrates the agentic AI principles studied in CIS 600 including goal-directed behavior, tool use, shared state management, adaptive feedback, and multi-agent coordination and we are on track for a complete and evaluated system by the final submission deadline.

References.

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv:2210.03629, 2022.

[2] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," arXiv:2304.03442, 2023.

[3] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," arXiv:2303.11366, 2023.

[4] M. Wooldridge, An Introduction to Multi-Agent Systems, 2nd ed. Wiley, 2009.

[5] W. Sun, Z. Shi, S. Gao, J. Ren, P. de Rijke, and Z. Ren, "A Survey on Large Language Model based Autonomous Agents," IEEE Access, 2024.

[6] LangChain AI, "LangGraph Documentation," https://langchain-ai.github.io/langgraph/, 2024.